CHAPTER 3 · IESTI05 EDGE AI ENGINEERING

Multimodal AI at the Edge

Giving a small model eyes

> One extra file · and the same board describes a photograph

Marcelo Rovai · UNIFEI

AIEng4D

THE MOTIVATION

Text is a strange interface for a visual problem.

Water sitting in an old tire. A plant saucer that never drains. A bucket left open after the rain. The thing you want to catch is visual.

THE ROUTE

Seven stages, one seeing board at the end

1	**Projector**	One extra file turns a text model into a vision model. What is inside it.
2	**Serving**	Two files that must match, one weight format, four flags that decide if it works.
3	**The API**	The WebUI is the smoke test. For repeatable work, base64 data URIs.
4	**CPU limits**	Why it runs forever, why it freezes at 2%, and how to read free -h calmly.
5	**0.8B or 2B**	Both measured on the same image. A real trade, not a free upgrade.
6	**Experiment**	Can an un-tuned 0.8B judge whether a photo shows a breeding site?
7	**Further**	A live camera, a bootstrapped dataset, and the hand-off into Chapter 4.

STAGE 1 OF 7

The projector

1

One extra file turns a text model into a vision model.

ARCHITECTURE

Not vision bolted on — trained to see

THE OLD RECIPE

LLaVA, Moondream

A frozen language model, a frozen image encoder, and a separately trained adapter glued between them.

Vision is an attachment.

QWEN3.5

A unified foundation

Trained on text and image tokens together from the start — early fusion. Alignment is learned jointly with the rest of the network, not bolted on afterwards.

This is also why Qwen3.5 beats the earlier Qwen3-VL models rather than inheriting from them — a different family, not a newer version.

ARCHITECTURE

The split is packaging, not architecture

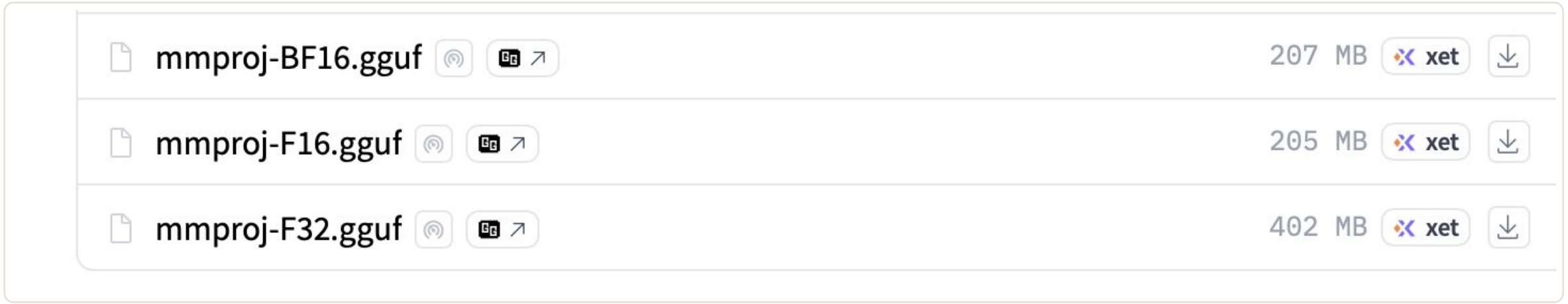


image → vision encoder → projector → language model → text

The -m file holds the language model. The mmproj holds the vision encoder and the connector. Run without the mmproj and you skip the vision pathway entirely — which is exactly what Chapter 2 did.

THE PIPELINE

What happens to an image at inference

1 **Vision encoder**
The image is chopped into patches; a ViT returns one feature vector per patch, in its own space — unrelated to the language model's vocabulary.

2 **Multimodal connector**
It reshapes those vectors to the model's embedding dimension, so a patch of fur behaves the way the text embedding for 'fur' would.

3 **Spliced into the token stream**
Inserted where an image placeholder sits. There is no 'image mode' — the picture has become something that reads like tokens.

4 **Language model**
It continues the sequence, conditioned on embeddings it can now read. That is why 'describe this image' produces a caption.

> The vision pathway produces no text — only embeddings. Every word in the answer comes from the language model.

THE CONSEQUENCE

An image is not one token — it is hundreds

hundreds to thousands

visual tokens per image, depending on resolution

-c 4096

the context size this chapter needs

--image-max-tokens

the dial that caps the encoder's token budget

The -c 1024 that was fine for the text classifier in Chapter 2 will overflow before you have even added a prompt.

STAGE 2 OF 7

Loading and serving

2

Two files that must match, one weight format that matters, and four flags.

THE PAIRING RULE

The model and its projector must match.

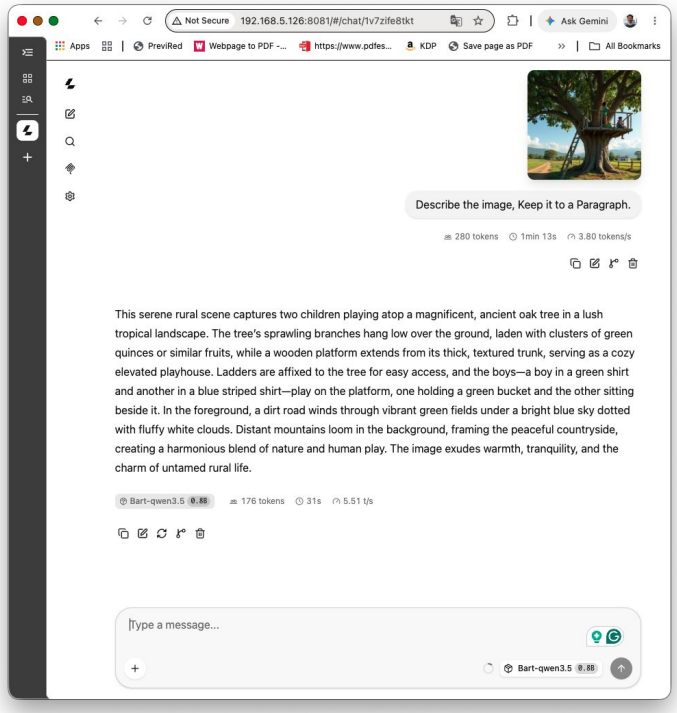
Their embedding dimensions have to agree. Pair a projector with the wrong base and llama-server refuses to load, with an n_embd mismatch. If you took the model from Bartowski, take the projector from Bartowski too.

GETTING IT

Free the port, then download the projector

```
# the Chapter 2 server is holding 8081 — stop it
$ sudo ss -ltnp | grep 8081     # should print nothing
$ free -h

$ cd ~/models/Qwen3.5-0.8B-GGUF
$ wget -c "https://huggingface.co/unsloth/\
   Qwen3.5-0.8B-GGUF/resolve/main/mmproj-F16.gguf"
```

> Stop the old server first — it is holding both the port and RAM you are about to need.



The files listed on the Hugging Face model page

WEIGHT FORMAT

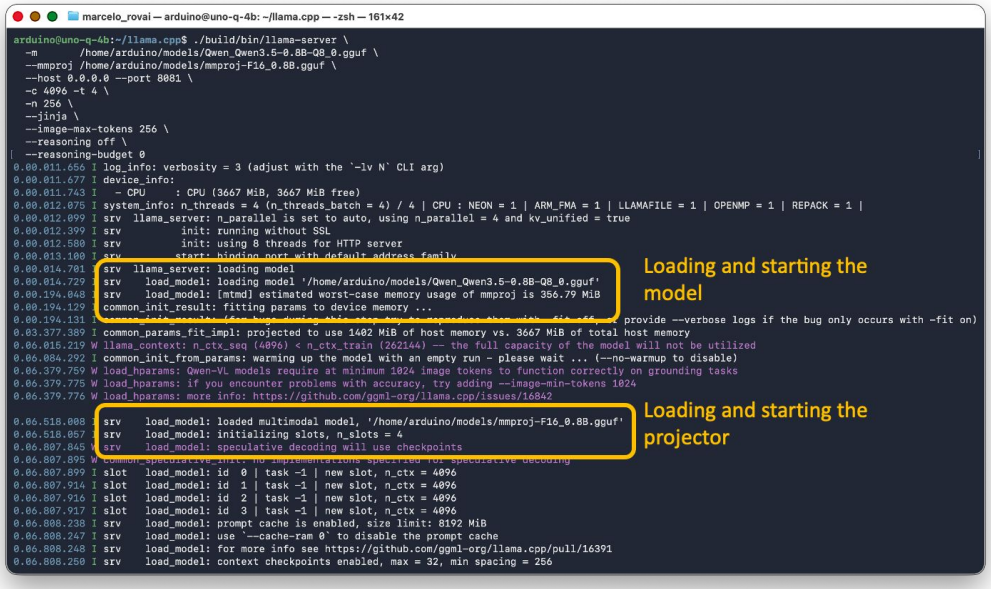
Prefer F16, not BF16.

The A53 cores have no native BF16 path, so BF16 weights get converted on the fly — wasting memory bandwidth you cannot spare. Switching dropped the description time for a 640×640 image from 4 minutes to about 1.

SERVING

The vision server

```
$ ./build/bin/llama-server \
  -m        ~/models/Qwen_Qwen3.5-0.8B-Q8_0.gguf \
  --mmproj ~/models/mmproj-0.8B-f16.gguf \
  --host 0.0.0.0 --port 8081 \
  -c 4096 -t 4 \
  -n 256 \
  --jinja \
  --image-max-tokens 256 \
  --reasoning off --reasoning-budget 0 \
  --alias qwen3.5-0.8b
```



The clip/mtmd block confirms the projector loaded

> If a request returns "image processing requires a vision model", the mmproj did not load. Nothing else produces that message.

SERVING

The four flags that actually matter

1

--jinja

Turns on the model's own chat template. Skip it and generation never stops — the fallback template does not register Qwen's end-of-turn token as a stop string.

2

-c 4096

The context has to hold hundreds of visual tokens before your prompt even starts.

3

--image-max-tokens 256

Caps the encoder's token budget. This is your main speed dial — Section 4 comes back to it.

4

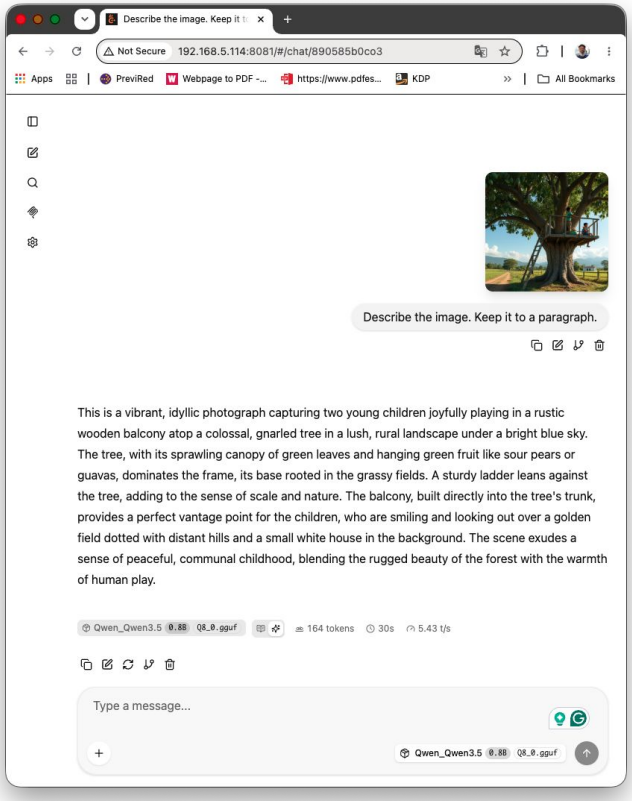
-n 256

Caps output length as a safety net. Section 6 revisits this: it interacts with the thinking trace in a way that surprises people.

--host 0.0.0.0 reaches the WebUI from a laptop. --alias keeps a stable model name, whichever GGUF is loaded.

FIRST CONTACT

Drop an image into the WebUI



A first description, from the embedded WebUI

> Keep images under 640×640 — Section 4 explains why that number matters so much.

FIRST CONTACT

What a small VLM gets right, and wrong

THE GIST

Genuinely good

On a generated scene it named the platform, the kids, the ladder, the path, the clouds and the mountains.

That is the judgement a breeding-site screen actually needs.

THE SPECIFICS

Unreliable

Then it called the hanging papayas 'quinces or similar fruits'.

It can tell you 'there is a container with standing water'. Do not trust it to identify a specific 200-litre drum.

Design the question around what it is actually good at.

STAGE 3 OF 7

The API

3

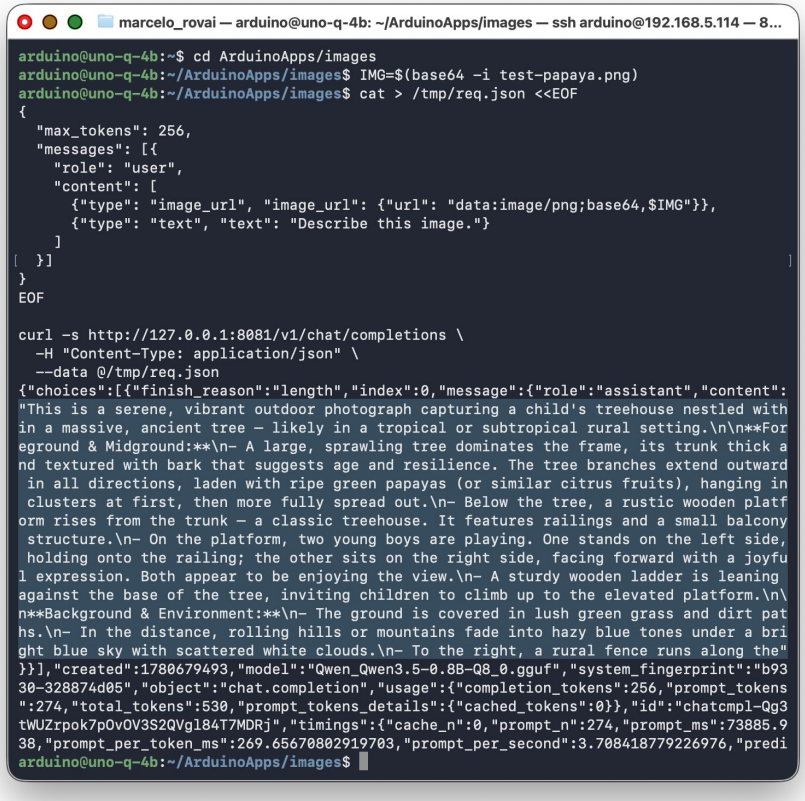
The WebUI is the smoke test. For anything repeatable, send the image as a base64 data URI.

TWO WAYS IN

curl — write the body to a file

```
$ IMG=$(base64 -w 0 test-papaya.png)

$ cat > /tmp/req.json <<EOF
{ "max_tokens": 256,
  "messages": [{"role": "user", "content": [
    {"type": "image_url", "image_url":
      {"url": "$IMG"}},
    {"type": "text", "text": "Describe this."}]}]}
EOF

$ curl -s http://127.0.0.1:8081/v1/chat/completions \
    -H "Content-Type: application/json" \
    --data @/tmp/req.json
```

Match the media type to the file: image/png for .png, image/jpeg for .jpg.



The smoke test

THE TRAP

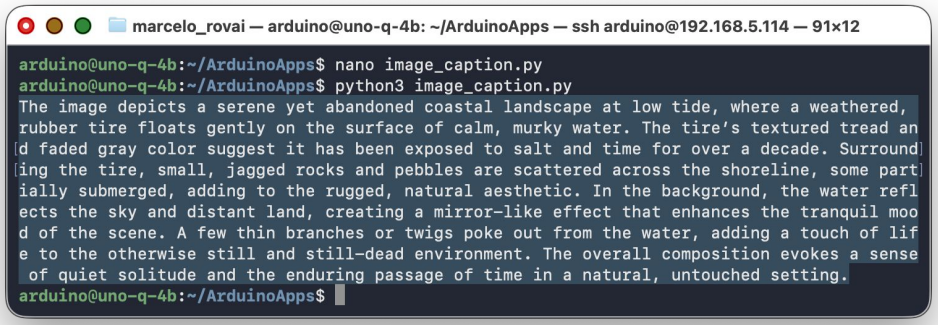
Never paste base64 into a curl -d argument.

A base64 image is large enough to exceed the shell's argument-size limit, and you get "Argument list too long" before the request is even sent. Write the body to a file, or pipe it on stdin, so it never goes through the command line.

TWO WAYS IN

Python — for anything real

```
def describe_image(path, prompt="Describe this image.",
                   max_tokens=256):
    uri = f"{encode_image(path)}"
    payload = {"messages": [{"role": "user", "content": [
        {"type": "image_url", "image_url": {"url": uri}},
        {"type": "text",  "text": prompt}]}],
        "temperature": 0.7, "max_tokens": max_tokens}

    # 300 s: a vision request takes 1-2 minutes here
    r = requests.post(SERVER, json=payload, timeout=300)
    r.raise_for_status()
    return r.json()["choices"][0]["message"]["content"]
```



describe_image() in use

> timeout=300 is not paranoia. The default gives up long before the model does.

STAGE 4 OF 7

CPU realities

4

Why it never stops, why it freezes at 2%, and how to read free -h without scaring yourself.

TWO SYMPTOMS

The failures that eat an afternoon

SYMPTOM 1

It runs forever

The fix is --jinja plus the -n cap.

If you have both and it still runs away, the WebUI's own max-tokens setting is overriding the server's -n. Lower it in the sampler panel too.

SYMPTOM 2

It freezes at 2%

A tall collage parked the progress bar at 2% with an ETA of 113 seconds. Ten minutes later: still 2%.

It was not stuck. It was facing tens of minutes of honest math.

On this board, vision is compute-bound on image size — not memory-bound at sane sizes.

READING FREE -H CORRECTLY

83 MiB free is not what it looks like.

llama-server memory-maps the GGUF, so the weights appear under buff/cache — 2.2 GiB — not under used. The number that matters is available: 2.1 GiB free, swap untouched. The 2B fits comfortably. It is just slow.

OPTIMISATION

The speed levers, in order of payoff

1

Cap the encoder's token budget

--image-max-tokens 256 · encode time scales with token count, so capping it directly reduces the work. Reach for this first.

2

Downscale the input

Feed it ~448–512 px. Complementary, not redundant — one bounds what the encoder produces, the other what it starts from.

3

Keep the output short

Every generated token costs real time. Same discipline as Chapter 2: wall-clock latency is what people feel.

4

Rebuild with OpenBLAS

-DGGML_BLAS=ON · encode and prefill are matrix-multiply heavy. Measure before and after — the gain on ARM varies.

One more, unrelated to speed: drop --cache-prompt for image workloads. Each image adds 1000+ KV tokens, it grows without bound, and the process eventually gets OOM-killed.

STAGE 5 OF 7

0.8B or 2B?

5

Both measured on the same 640×640 image. A real trade, not a free upgrade.

MEASURED

The same image, both models

	Qwen3.5-0.8B (Q8_0)	Qwen3.5-2B (UD-Q4_K_XL)
Projector (F16)	~220 MB	668 MB
Decode speed	5.42 tok/s	2.44 tok/s
Time for one describe	~1 min	~4 min
Peak RAM (mmap included)	well under the ceiling	~3 GB — fits, swap ~0
Description quality	good gist, shaky specifics	noticeably better

> Treat them as two tiers, not two options. Five minutes per image rules out anything interactive.

NOT OPTIONAL

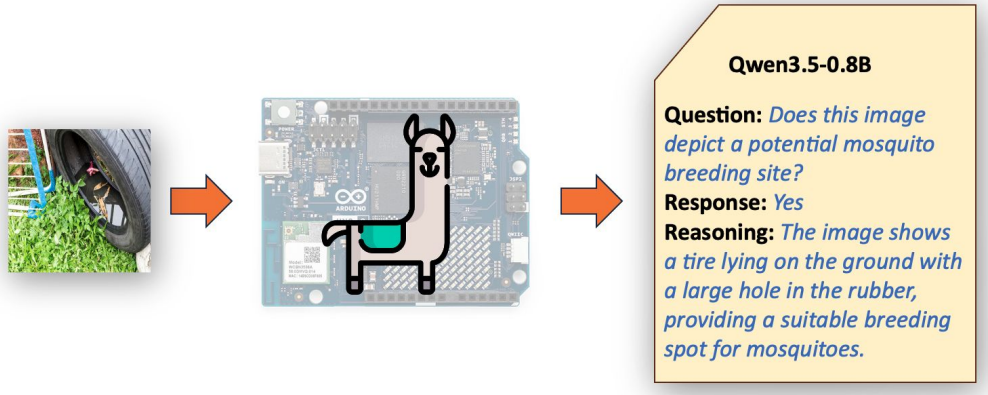
The bigger model needs its bigger projector.

668 MB versus 220 MB. The small one will not match — the load fails with an n_embd mismatch. It is three times larger because the vision encoder inside it is bigger, which is also why encoding is slower: every patch passes through a heavier tower.

IN PRACTICE

One server at a time, same port

```
#!/usr/bin/env bash   # ~/run-qwen-0.8b-vision.sh
set -e
exec ~/llama.cpp/build/bin/llama-server \
  -m       ~/models/Qwen_Qwen3.5-0.8B-Q8_0.gguf \
  --mmproj ~/models/mmproj-F16_0.8B.gguf \
  --host 0.0.0.0 --port 8081 -c 4096 -t 4 -n 256 \
  --jinja --image-max-tokens 256 \
  --reasoning off --reasoning-budget 0 \
  --alias qwen3.5-0.8b
```



Two scripts, one port

Distinct aliases are deliberate: only one server runs at a time, so the alias echoed back in the response confirms the swap actually took effect.

IN PRACTICE

Run it in tmux

```
$ tmux new -s vlm
$ ~/run-qwen-0.8b-vision.sh

# Ctrl+B then D to detach
$ tmux attach -t vlm      # to return
```

> A describe call can take several minutes. You do not want a dropped SSH connection to kill it.

THE RULE OF THUMB

Use the smallest model that meets the task.

The 0.8B is the right default for anything interactive or run over many frames. Reach for the 2B only when a specific case needs the reasoning quality and the extra minutes are acceptable.

STAGE 6 OF 7

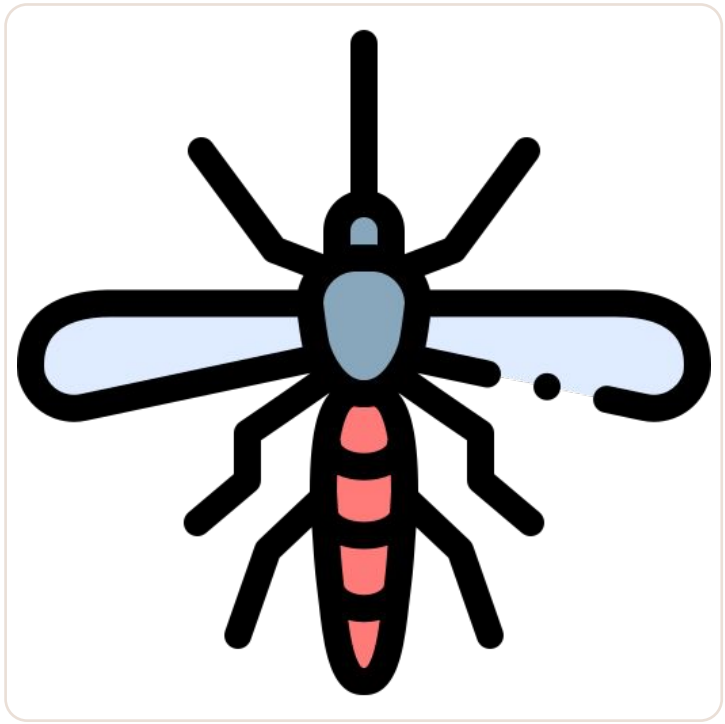
The experiment: VisText-Mosquito

6

Can an un-tuned 0.8B reason about whether a photo shows a mosquito breeding site?

THE TASK

The prompt, straight from the paper



VisText-Mosquito: detection, segmentation and reasoning about breeding sites

Their fine-tuned Mosquito-LLaMA3-8B reached a final loss of 0.0028 and a BLEU of 54.7. That is the ceiling we are measuring against.

THE TASK

The prompt from the paper

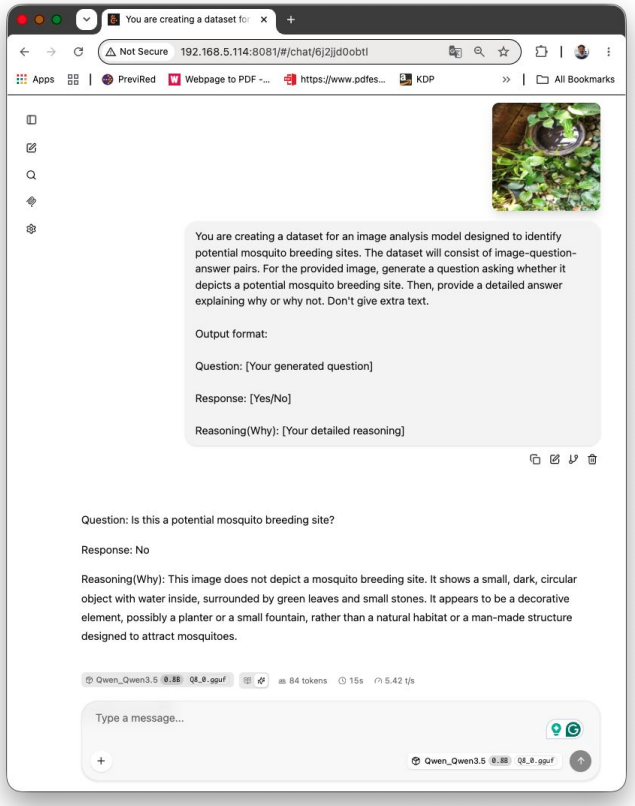
```
You are creating a dataset for an image analysis model
designed to identify potential mosquito breeding sites.
For the provided image, generate a question asking
whether it depicts a potential breeding site. Then
provide a detailed answer explaining why or why not.
Don't give extra text.

Output format:
Question: [Your generated question]
Response: [Yes/No]
Reasoning(Why): [Your detailed reasoning]
```

Note the explicit output structure. Small models follow structure far better than they follow instructions.

One image from the dataset

ATTEMPT ONE

Right structure, wrong verdict



It understood the question, understood the image, followed the format — and got it wrong

> The image IS a potential breeding site. Everything worked except the one thing that mattered.

ATTEMPT TWO

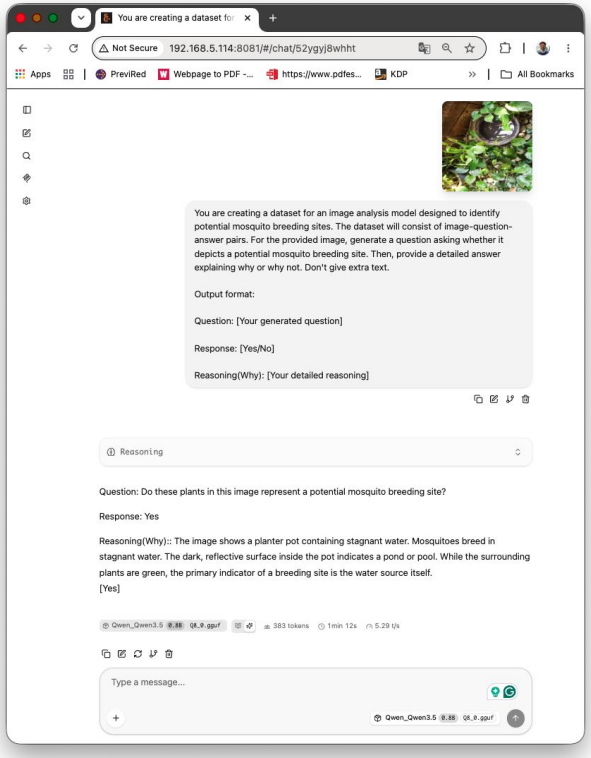
The -n 256 trap

```
$ ./build/bin/llama-server \
  -m       ~/models/Qwen_Qwen3.5-0.8B-Q8_0.gguf \
  --mmproj ~/models/mmproj-F16_0.8B.gguf \
  --host 0.0.0.0 --port 8081 -c 4096 -t 4 \
  -n 1024 \        # room for thinking AND the answer
  --jinja --image-max-tokens 256 \
  --alias qwen3.5-0.8b     # note: no --reasoning off
```

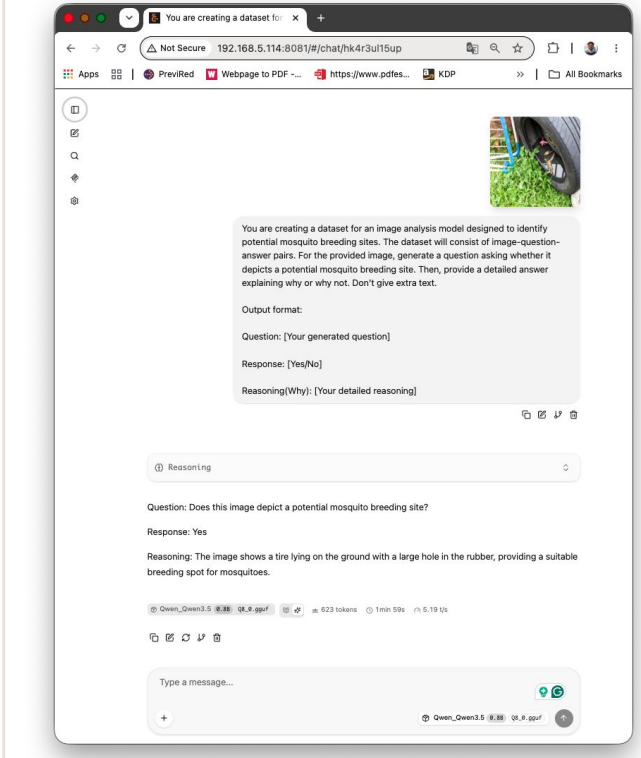
With reasoning ON, the <think> trace is written FIRST. The -n 256 cap was fully consumed by it, so the model never reached the structured output that was actually asked for.

ATTEMPT THREE

Correct — and you can read why



The bucket: a correct Yes



An old tire: correct again

THE COST OF THINKING

1′20″
start + image processing

1′00″
reasoning

0′20″
writing, at 5.3 tok/s

STAGE 7 OF 7

Going further

7

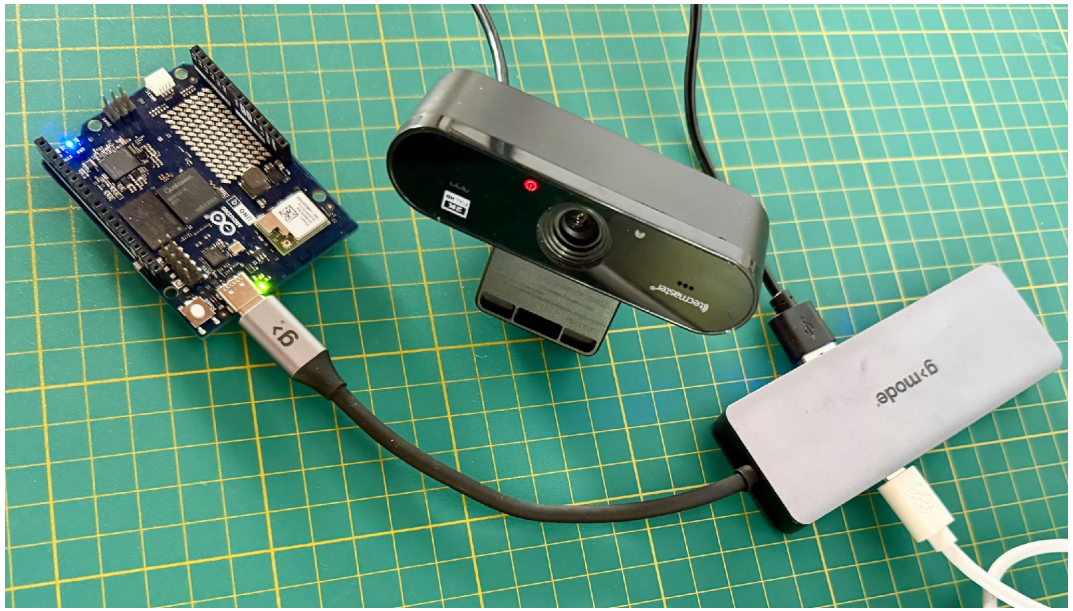
A live camera, a bootstrapped dataset, and the hand-off into the dual-brain project.

DIRECTION 1

Feed it a live camera

```
$ sudo halt                    # power down before rewiring

# hub → power supply, webcam → hub, UNO Q → hub

$ sudo apt install -y fswebcam
$ fswebcam -r 640x480 --skip 20 --no-banner \
           ~/ArduinoApps/images/frame.jpg
```

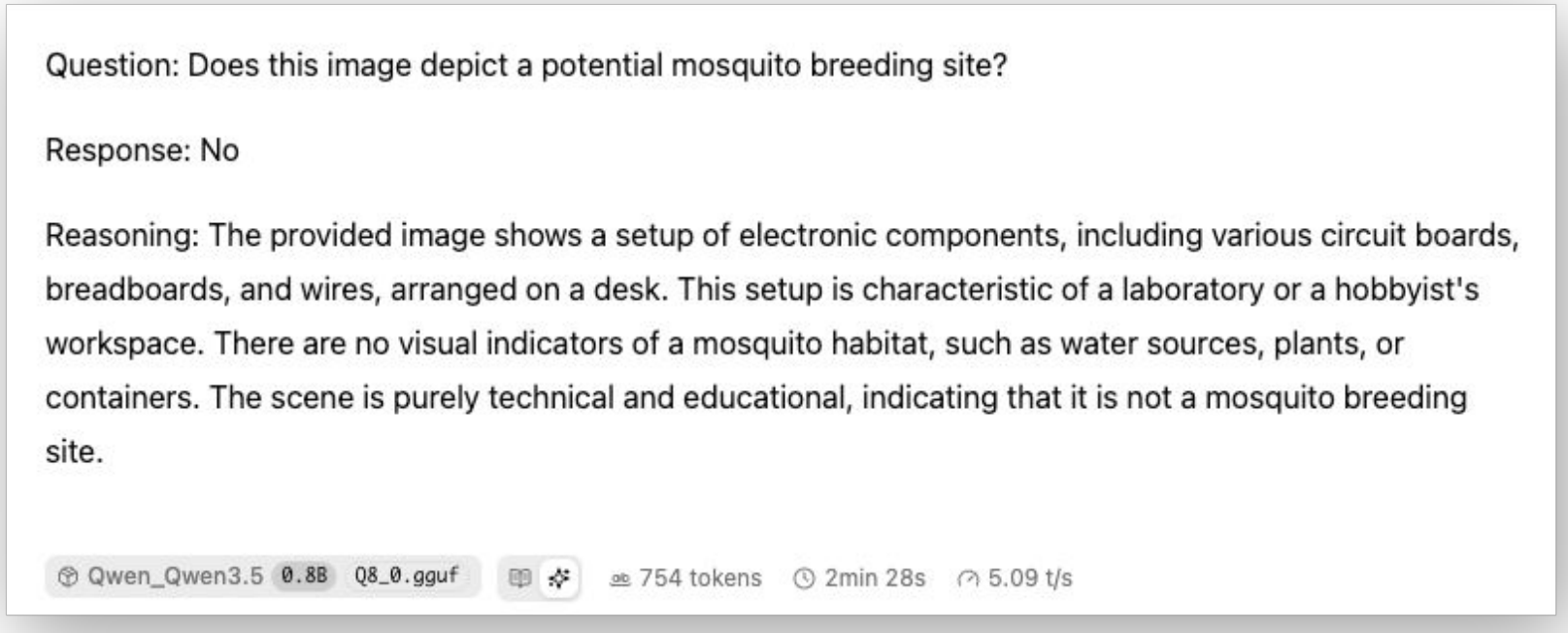
640×480 is not a coincidence — it is the resolution lever from Section 4, applied at capture time rather than after.



Hub, webcam, board

DIRECTION 1

A frame from the desk, described on the board

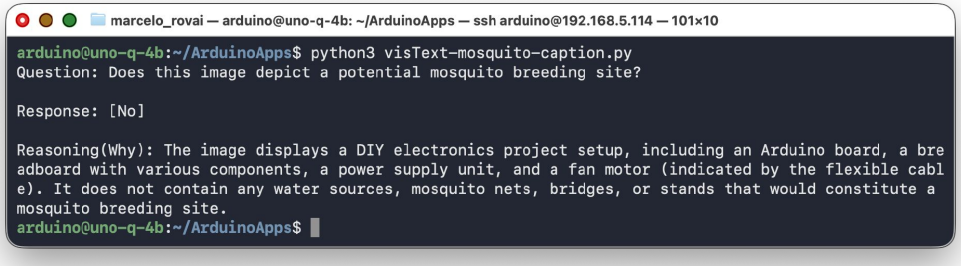


Live camera in, description out — same describe_image(), different source

DIRECTION 2

Bootstrap a labelled dataset overnight

```
import json, glob

with open("labels.jsonl", "w") as out:
    for path in sorted(glob.glob("images/field/*.jpg")):
        verdict = describe_image(path,
                                 prompt=MOSQUITO_PROMPT,
                                 max_tokens=1024)
        out.write(json.dumps({"image": path,
                              "verdict": verdict}) + "\n")
        print(f"done: {path}")
```



Running over a folder

You cannot ship the labels unread. Treat the output as a first draft a human curates — your reviewer is correcting, not authoring, and that is most of the cost gone.

ZERO-SHOT AND FINE-TUNED

Not rivals

WHAT WE DID

Zero-shot

A general model reasoning about a task it was never trained for. Cheap, immediate, and good enough to bootstrap.

WHAT THE PAPER DID

Fine-tuned

An 8B model trained on the dataset, landing a BLEU of 54.7 — well above what an un-tuned 0.8B reaches.

THE RELATIONSHIP

Sequential

Zero-shot bootstraps the dataset cheaply. Fine-tuning turns it into a specialist. One feeds the other.

Direction 3 — Chapter 4 wires in the MCU: sensors, Bridge RPC, an RGB LED reporting a verdict. A button stands in for a water sensor; with this chapter's camera, that input can come from the model instead.

CONCLUSION

What works, and what does not

IT WORKS

Vision runs here

A coin-sized CPU describes a scene and reasons about it, offline. A couple of years ago that sentence would have been a joke.

The gist is reliable — and one model, two modes: the mmproj is the only addition.

IT IS SLOW

A batch tool, not a chat

A minute to a few minutes per image rules out anything interactive.

Compute-bound, not memory-bound: a too-large image does not crash the board, it makes you wait ten minutes for an answer you could have had in one.

IT IS UNRELIABLE

On specifics

It called papayas 'quinces'. It will name the wrong container, invent a detail, occasionally flip a verdict.

Trust the gist, never the specifics.

THE TAKEAWAY

The interesting use is not real-time vision.

It is dataset bootstrapping. A board this cheap, generating first-draft labels offline, turns the expensive part of a supervised pipeline into something one field device does overnight.

FIVE THINGS TO REMEMBER

From Chapter 3

1. **The mmproj must match the base**
   Wrong pairing means an n_embd mismatch and no load at all.
2. **Prefer F16 over BF16**
   No native BF16 path on the A53s. This was 4 minutes versus 1.
3. **Give it -c 4096**
   An image is hundreds of visual tokens before your prompt starts.
4. **Use --jinja, or it never stops**
   The fallback template does not know Qwen's end-of-turn token.
5. **Resolution is the biggest lever**
   Vision here is compute-bound on image size.

A note on the VENTUNO Q: 16 GB and a Dragonwing NPU make the 2B interactive instead of a five-minute wait. Same mmproj split, same flags — only the clock changes.

RESOURCES

Where to read further

Resource	Where
`llama.cpp multimodal (mtmd) docs`	github.com/ggml-org/llama.cpp/blob/master/docs/multimodal.md
`llama.cpp mtmd / libmtmd README`	github.com/ggml-org/llama.cpp/blob/master/tools/mtmd/README.md
`Qwen3.5-0.8B GGUF + mmproj (Unsloth)`	huggingface.co/unsloth/Qwen3.5-0.8B-GGUF
`ggml-org multimodal GGUF collection`	huggingface.co/collections/ggml-org/multimodal-ggufs
`VisText-Mosquito paper (arXiv)`	arxiv.org/abs/2506.14629
`VisText-Mosquito dataset (Mendeley)`	data.mendeley.com/datasets/rtsfh7jh7p/3

Next — Ch. 4, GenAI Meets the Real World: a dual-brain dengue-risk classifier. The MCU senses, the MPU reasons — and after this chapter, the reasoning can include sight.

Questions?

Prof. Marcelo J. Rovai

rovai@unifei.edu.br

UNIFEI — Federal University of Itajubá, Brazil

AiEng4D — Academic Network Co-Chair

EdgeAIP — Academia-Industry Partnership Co-Chair